

Week 7 Cognizant Mandatory Hands On

1. Single-Page Application (SPA) vs. Multi-Page Application (MPA)

What is an SPA?

A Single-Page Application (SPA) is a web application that fits on a single web page. Instead of loading entire new HTML pages from the server upon every user interaction, an SPA dynamically updates the current page content using JavaScript.

Key Benefits of SPAs

- Fluid User Experience: Feels like a desktop application because pages do not refresh constantly.
- Fast Navigation: Once the initial HTML/JS bundle is loaded, swapping views requires only small data fetches (JSON via APIs).
- Caching Capability: SPAs can effectively cache local data and state, allowing smooth performance and even partial offline functionality.

SPA vs. MPA: Core Differences

Feature	Single-Page Application (SPA)	Multi-Page Application (MPA)
Page Reloads	Loads once; dynamic UI updates without full reloads.	Reloads the entire page on every request/navigation.
Speed & Performance	Slower initial load, extremely fast	Faster initial page load, but slower page transitions.

Feature	Single-Page Application (SPA)	Multi-Page Application (MPA)
	subsequent updates.	
Architecture	Client-side rendering (CSR) heavy.	Server-side rendering (SSR) heavy.
SEO	Requires extra configuration (like SSR/SSG frameworks).	SEO-friendly out of the box due to individual server-rendered pages.
Best For	Dashboards, social media apps, SaaS tools (e.g., Gmail, Trello).	E-commerce sites, news blogs, large multi-category portals.

Pros & Cons of Single-Page Applications

- Pros:
 - Fast rendering after initial load.
 - Clear separation between frontend (UI) and backend (API services).
 - Easy state management across user actions without losing context.
- Cons:
 - Longer initial load time due to loading large JavaScript bundles.

- SEO challenges if content isn't pre-rendered.
- Requires client-side memory management to prevent memory leaks.

2. Fundamentals of React

What is React?

React is an open-source JavaScript library developed by Meta (Facebook) for building user interfaces—specifically component-based, interactive single-page web applications.

How React Works

React operates around the concept of Components (reusable blocks of UI like buttons, forms, or headers). It follows a declarative approach: you describe what the UI should look like for a given state, and React handles rendering and updating the DOM automatically when data changes.

The Virtual DOM

Updating the real Browser DOM is slow. To solve this, React uses a Virtual DOM (VDOM):

1. In-Memory Representation: React maintains a lightweight virtual copy of the actual DOM in memory.
2. Reconciliation (Diffing): When state changes occur, React creates a new Virtual DOM tree and compares it with the previous Virtual DOM tree to identify exactly what changed.
3. Batch Updates: React updates only the modified nodes in the real DOM, avoiding expensive, full-tree re-renders.

Key Features of React

- Component-Based Architecture: Encapsulated components manage their own state and can be composed to build complex UIs.

- JSX (JavaScript XML): A syntax extension allowing HTML-like markup directly inside JavaScript code.
- Unidirectional Data Flow: Data flows downward from parent components to child components via props, making code predictable and easier to debug.
- Rich Ecosystem: Large community support, custom hooks, and rich toolchains.

3. Hands-on Lab: Setting Up a React Environment

1.1. Verify Prerequisites:Terminal / Command Prompt.

Open your terminal or command prompt and check if Node.js and NPM are installed:

Bash

```
node -v
```

```
npm -v
```

If both print version numbers (e.g., Node v18.x.x or higher), you are ready to proceed. If not, download Node.js from nodejs.org.

2.2. Create a React Application:Using npx / create-react-app.

Run the following command to generate a template React project named my-first-app:

Bash

```
npx create-react-app my-first-app
```

(Note: npx comes bundled with NPM 5.2+ and runs create-react-app without requiring a permanent global installation).

3.3. Navigate and Open in VS Code:Terminal.

Move into your new project directory and open it inside Visual Studio Code:

Bash

```
cd my-first-app
```

```
code .
```

4.4. Start the React Development Server:VS Code Terminal.

Inside VS Code, open the integrated terminal (Ctrl + ~ or Cmd + ~) and start the app:

Bash

```
npm start
```

2 – Hands On

1. Core Concepts

Props vs. State

Feature	Props (Properties)	State
Definition	Read-only input passed from a parent to a child component.	Data managed internally within a component that can change over time.
Mutability	Immutable (cannot be changed by the receiving component).	Mutable (updated using <code>this.setState()</code> or hooks like <code>useState</code>).
Ownership	Owned by the parent component.	Owned by the component where it is declared.

Feature	Props (Properties)	State
Re-rendering	Changes in props cause the child component to re-render.	Updating state triggers a re-render of the component and its children.

Event Handling in React

Handling events in React is very similar to handling events on DOM elements, with two main differences:

1. **CamelCase Naming:** React events use camelCase (e.g., onClick, onChange, onSubmit).
2. **Function References:** You pass a function handler rather than a string (e.g., onClick={handleClick}).

2. Hands-on Lab: Student Details & Score Tracker

In this lab, you will pass student details as **props** from App.js to a Student component, and manage interactive score tracking using **state** and **event handlers**.

1.1. Create the Student Component (Using Props & State): Step 1.

Inside src/Components/, create a new file named Student.js. This component accepts name and rollNo as **props**, and tracks the student's score using **state**.

JavaScript

```
import React, { Component } from 'react';
```

```
class Student extends Component {
  constructor(props) {
```

```
super(props);  
// Initialize State  
this.state = {  
  score: 0  
};  
}  
  
// Event Handler Method  
increaseScore = () => {  
  this.setState({ score: this.state.score + 1 });  
};  
  
render() {  
  return (  

```

Student Profile

Name: {this.props.name}

Roll No: {this.props.rollNo}

Current Score: {this.state.score}

{/* Event Handler Binding */}

+1 Increment Score

```
);  
}  
}
```

export default Student;

2.2. Pass Props from App.js:Step 2.

Update src/App.js to import and render multiple instances of the Student component with different props:

JavaScript

```
import React, { Component } from 'react';
```

```
import Student from './Components/Student';
```

```
class App extends Component {
```



```
render() {  
  return (  

```

Student Management Portal - Score Tracker

```
{/* Passing attributes as Props */}
```

```
);  
}  
}
```

```
export default App;
```

3.3. Run and Test:Step 3.

Start the app using your terminal:

Bash

```
npm start
```

Open `http://localhost:3000` in your browser. Click the **" +1 Increment Score"** button for each student to observe how state updates independently while receiving static details via props.

3 – HANDS ON

Key Concepts

Function Components vs. Class Components

- **Function Components:** Pure JavaScript/ES6 functions that accept props as an argument and return JSX. They are concise, easier to write, and are the modern standard in React development.
- **Styling in React:** Can be applied via **External Stylesheets** (importing a .css file using standard CSS classes mapped via `className`) or **Inline Styles** (passing a JavaScript object to the `style` attribute).

Hands-on Lab: Score Calculator App

1.1. Create the React App: Step 1.

Open your VS Code terminal and create the new project:

Bash

```
npx create-react-app scorecalculatorapp
```

```
cd scorecalculatorapp
```

2.2. Create the CalculateScore Functional Component: Step 2 & 3.

Inside `src/`, create a new folder named `Components`. Inside `src/Components/`, create `CalculateScore.js` and add the following functional component code:

JavaScript

```
import React from 'react';
```

```
import '../Stylesheets/mystyle.css';

// Function component accepting props
function CalculateScore({ Name, School, Total, Goal }) {
  // Logic to calculate average
  const average = (Total / 5).toFixed(2);

  return (
```

Student Details:

Name: {Name}

School: {School}

Total: {Total} Marks

Score: {average} %

Goal: {Goal}

```
);  
}
```

```
export default CalculateScore;
```

3.3. Create the Stylesheet:Step 4.

Inside `src/`, create a new folder named `Stylesheets`. Inside `src/Stylesheets/`, create a file named `mystyle.css` with the following CSS styling:

CSS

```
.formatstyle {  
  background-color: lightgray;  
  width: 400px;  
  padding: 20px;  
  margin: 20px auto;  
  border: 2px solid darkgray;  
  border-radius: 10px;  
  font-family: Arial, sans-serif;  
}
```

```
.Name, .School, .Total, .Score, .Goal {  
  color: blue;  
  margin-bottom: 10px;  
  font-size: 16px;  
}
```

```
.Name span, .School span, .Total span, .Score span, .Goal span {  
  color: black;  
}
```

4.4. Update App.js:Step 5.

Update src/App.js to invoke CalculateScore with student details:

JavaScript

```
import React from 'react';
```

```
import CalculateScore from './Components/CalculateScore';
```

```
function App() {
```

```
  return (
```

```
  );
```

```
}
```

```
export default App;
```

5.5. Run and Verify:Steps 6 & 7.

Run the application in your terminal:

Bash

```
npm start
```

Navigate to <http://localhost:3000> in your browser. You will see the styled student card displaying the calculated average score (90.00 %).

4 – HANDS ON

1.1. Create the React App & Open in VS Code:Step 1 & 2.

Open your terminal and run:

Bash

```
npx create-react-app blogapp
```

```
cd blogapp
```

```
code .
```

2.2. Create Post Model Class (`src/Post.js`):Step 3.

Create a new file named Post.js inside the src/ directory to define the Post data model:

JavaScript

```
class Post {  
  constructor(userId, id, title, body) {  
    this.userId = userId;  
    this.id = id;  
    this.title = title;  
    this.body = body;  
  }  
}
```

```
export default Post;
```

3.3. Create Posts Component with Lifecycle Methods (`src/Posts.js`):Steps 4 to 9.

Create a new file named Posts.js in src/. This component initializes state, fetches data inside componentDidMount(), handles rendering, and catches errors using componentDidCatch():

JavaScript

```
import React, { Component } from 'react';  
import Post from './Post';
```

```
class Posts extends Component {
  constructor(props) {
    super(props);
    // Step 5: Initialize component state
    this.state = {
      posts: [],
      hasError: false,
      errorMessage: ''
    };
  }

  // Step 6: Load posts using Fetch API
  loadPosts() {
    fetch('https://jsonplaceholder.typicode.com/posts')
      .then((response) => {
        if (!response.ok) {
          throw new Error('Failed to fetch posts');
        }
        return response.json();
      })
      .then((data) => {
        // Map raw JSON objects to Post class instances
        const postList = data.map(
          (item) => new Post(item.userId, item.id, item.title, item.body)
        );
      })
  }
}
```



```
);  
  this.setState({ posts: postList });  
})  
.catch((error) => {  
  alert('Error loading posts: ' + error.message);  
  this.setState({ hasError: true, errorMessage: error.message });  
});  
}
```

```
// Step 7: Call loadPosts() inside componentDidMount()  
componentDidMount() {  
  this.loadPosts();  
}
```

```
// Step 9: Define componentDidCatch() to capture unhandled  
component errors  
componentDidCatch(error, info) {  
  alert('An unexpected error occurred in Posts component: ' +  
error.toString());  
  console.error('Error info:', info);  
  this.setState({ hasError: true, errorMessage: error.toString() });  
}
```

```
// Step 8: Render the list of titles and body text
```

```
render() {  
  if (this.state.hasError) {  
    return
```

Something went wrong while displaying posts.

```
;  
  }
```

```
  return (
```

Blog Posts

```
{this.state.posts.length === 0 ? (
```

Loading posts...

```
) : (  
  this.state.posts.map((post) => (
```

```
    {post.id}, {post.title}
```

```
{post.body}
```

```
    ))  
  }}
```

```
  );  
}  
}
```

```
export default Posts;
```

4.4. Update App Component (`src/App.js`):Step 10.

Replace the contents of src/App.js to render the Posts component:

JavaScript

```
import React from 'react';
```

```
import Posts from './Posts';
```

```
function App() {
```

```
  return (
```

```
);  
}
```

```
export default App;
```

5.5. Run and Verify: Step 11.

In your terminal, execute:

```
Bash
```

```
npm start
```

5 – HANDS ON

Key Concepts

Why Style React Components?

Styling enhances user experience and visual hierarchy. In React, there are multiple ways to apply styles:

- **Inline Styles:** Passed directly as JavaScript objects via the style prop. Useful for dynamic styling based on conditions (e.g., changing colors based on props or status).
- **CSS Modules:** Saved as [filename].module.css. React scopes class names locally to prevent global CSS selector collisions across components.

Hands-on Lab: Cohort Details Dashboard

1.1. Setup and Package Restoration: Steps 1 to 4.

Unzip your project, navigate to the project directory in your terminal, and install the dependencies:

Bash

```
npm install
```

Then open the project in Visual Studio Code:

Bash

```
code .
```

2.2. Create the CSS Module (`CohortDetails.module.css`): Steps 5 to 7.

Inside the `src/` directory (or inside your component folder, e.g., `src/components/`), create a new file named `CohortDetails.module.css` and add the following CSS rules:

CSS

```
/* Step 6: Class definition for .box */
```

```
.box {
```

```
  width: 300px;
```

```
  display: inline-block;
```

```
  margin: 10px;
```

```
  padding: 10px 20px;
```

```
  border: 1px solid black;
```

```
  border-radius: 10px;
```

```
}
```

```
/* Step 7: Element selector for
*/
dt {
  font-weight: 500;
}
```

3.3. Update `CohortDetails.js` Component: Steps 8 to 10.

Open CohortDetails.js (or CohortDetails.jsx), import the CSS Module, apply the .box class using className, and use dynamic inline styling for the <h3> header based on the cohort status:

JavaScript

```
import React from 'react';

// Step 8: Import the CSS Module
import styles from './CohortDetails.module.css';

function CohortDetails({ cohort }) {
  const { name, status, startDate, duration } = cohort;

  // Step 10: Dynamic inline style rule based on status
  const headerStyle = {
    color: status.toLowerCase() === 'ongoing' ? 'green' : 'blue'
  };

  return (
    /* Step 9: Apply the scoped .box class */
```

{/* Step 10: Apply dynamic inline style to h3 */}

{name}

Status:

{status}

Start Date:

{startData}

Duration:

```
{duration}
```

```
);  
}
```

```
export default CohortDetails;
```

4.4. Run the Application:Final Verification.

Start the local development server from your terminal:

Bash

```
npm start
```

9-HANDS ON

1. Core ES6 Concepts

Features of ES6 (ECMAScript 2015)

- Block-scoped variable declarations (let, const).
- Arrow functions (() => {}).
- Array destructuring and object destructuring.
- Spread operator (...) for array merging and cloning.
- Array iteration methods like map(), filter(), reduce().
- ES6 Classes and Inheritance (class, extends, super).

- Data structures like Set and Map.

Key Comparisons: var vs. let vs. const

Feature	var	let	const
Scope	Function scope	Block scope {}	Block scope {}
Re-declaration	Allowed in same scope	Not allowed in same scope	Not allowed in same scope
Re-assignment	Allowed	Allowed	Not allowed (constant reference)
Hoisting	Hoisted with undefined	Hoisted but in Temporal Dead Zone	Hoisted but in Temporal Dead Zone

Key Features Explained

- **ES6 Arrow Functions:** Provide a shorter syntax for writing functions and lexically bind this.
- **Array Destructuring:** Unpacks values from arrays into distinct variables (e.g., `const [first, second] = players`).
- **Spread Operator (...):** Expands an array/iterable into individual elements, ideal for merging arrays (`[...T20, ...Ranji]`).
- **map() and filter():** `map()` transforms every element in an array and returns a new array; `filter()` creates a new array containing elements that pass a test condition.

2. Hands-on Lab: Cricket App (cricketapp)

1.1. Create React Application:Step 1.

Open your terminal and run:

Bash

```
npx create-react-app cricketapp
```

```
cd cricketapp
```

2.2. Create ListofPlayers Component (`src/ListofPlayers.js`):Step 2.

Create a new file src/ListofPlayers.js. This component declares 11 players, displays all players using map(), and displays filtered players with scores ≥ 70 using filter() with arrow functions:

JavaScript

```
import React from 'react';
```

```
function ListofPlayers() {  
  // 1. Array of 11 players  
  const players = [  
    { name: "Jack", score: 50 },  
    { name: "Steve", score: 70 },  
    { name: "John", score: 40 },  
    { name: "Peter", score: 80 },  
    { name: "David", score: 90 },  
    { name: "Rahul", score: 60 },  
    { name: "Sachin", score: 95 },  
    { name: "Dhoni", score: 85 },  
    { name: "Virat", score: 92 },  
  ]  
}
```

```
{ name: "Rohit", score: 88 },  
{ name: "Kumble", score: 45 }  
];
```

```
// Filter players with scores >= 70 using arrow function  
const highScorers = players.filter((player) => player.score >= 70);  
  
return (
```

List of Players

```
{/* Map to render all players */}  
{players.map((item, index) => (  
  
    Mr. {item.name} {item.score}  
  
    )})}
```

List of Players having Scores 70 and Above

```
{/* Map to render high scorers */}
{highScorers.map((item, index) => (

    Mr. {item.name} {item.score}

    )))}

);
}
```

export default ListofPlayers;

3.3. Create IndianPlayers Component (`src/IndianPlayers.js`): Step 3.

Create a new file `src/IndianPlayers.js`. This component uses **Array Destructuring** for odd/even players and the **Spread Operator (...)** to merge T20 and Ranji Trophy arrays:

JavaScript

```
import React from 'react';

function IndianPlayers() {
  // Base array of players
  const indianTeam = ["Sachin", "Dhoni", "Virat", "Rohit", "Yuvraj",
    "Raina"];

  // Destructuring array to get individual players
  const [p1, p2, p3, p4, p5, p6] = indianTeam;

  // Grouping odd and even players
  const oddTeam = [p1, p3, p5];
  const evenTeam = [p2, p4, p6];

  // Arrays for merging
  const T20players = ["First Player", "Second Player", "Third Player"];
  const RanjiTrophy = ["Fourth Player", "Fifth Player"];

  // Merging using Spread Operator
  const mergedPlayers = [...T20players, ...RanjiTrophy];

  return (
```

Indian Players

Odd Team Players

```
{oddTeam.map((player, index) => (
```

```
First : {player}
```

```
)))}
```

Even Team Players

```
{evenTeam.map((player, index) => (
```

```
Second : {player}
```

```
)))}
```

List of Indian Players (Merged T20 & Ranji)

```
{mergedPlayers.map((player, index) => (  
  
  Mr. {player}  
  
  ))}  
  
);  
}
```

```
export default IndianPlayers;
```

4.4. Update App.js with Conditional Flag: Step 4.

Update src/App.js to conditionally switch between ListofPlayers and IndianPlayers using a simple flag variable:

JavaScript

```
import React from 'react';  
  
import ListofPlayers from './ListofPlayers';  
  
import IndianPlayers from './IndianPlayers';
```

```
function App() {
  // Set flag to true or false
  const flag = true;

  if (flag) {
    return (

  );
  } else {
    return (

  );
  }
}

export default App;
```


5.5. Run and Verify Outputs:Step 5.

Execute the app in your terminal:

Bash

npm start

- When flag = true, the browser displays **ListofPlayers** (all 11 players followed by high scorers ≥ 70).
- Change flag = false inside App.js and save; the browser updates to display **IndianPlayers** (Odd/Even destructured teams and merged T20/Ranji lists).

10- HANDS ON

1. CohortDetails.module.css (Steps 5–7)

CSS

```
/* Box class styling */
```

```
.box {  
  width: 300px;  
  display: inline-block;  
  margin: 10px;  
  padding: 10px 20px;  
  border: 1px solid black;  
  border-radius: 10px;  
}
```

```
/* Tag selector for <dt> */
```

```
dt {  
  font-weight: 500;  
}
```

2. CohortDetails.js (Steps 8–10)

JavaScript

```
import React from 'react';  
  
// Import CSS Module  
import styles from './CohortDetails.module.css';  
  
function CohortDetails({ cohort }) {  
  const { name, status, startData, duration } = cohort;  
  
  // Dynamic inline styling for <h3> based on status  
  const headerStyle = {  
    color: status.toLowerCase() === 'ongoing' ? 'green' : 'blue'  
  };  
  
  return (  
    /* Apply scoped .box class */  
    <div className={styles.box}>  
      /* Apply dynamic header style */  
      <h3 style={headerStyle}>{name}</h3>  
      <dl>  
        <dt>Status:</dt>
```

```

        <dd>{status}</dd>

        <dt>Start Date:</dt>
        <dd>{startData}</dd>

        <dt>Duration:</dt>
        <dd>{duration}</dd>
    </dl>
</div>
);
}

```

```
export default CohortDetails;
```

11-HANDS ON

Key Concepts

Event Handling & Synthetic Events in React

- **Multiple Functions in an Event Handler:** You can invoke multiple actions within a single event handler function (e.g., updating counter state and triggering an alert or message).
- **Passing Arguments to Handlers:** Use inline arrow functions like `onClick={() => handleWelcome('Welcome')}` to pass custom parameters to event handler functions.

- Synthetic Events: React wraps browser native events in a `SyntheticEvent` cross-browser wrapper to ensure consistent behavior across different web browsers.
- Controlled Forms: Form inputs are bound to React state (`onChange`), and submission is handled via `onSubmit` or `onClick` while preventing default browser reloads using `e.preventDefault()`.

Hands-on Lab: Event Examples App (eventexamplesapp)

1.1. Create the React App:Step 1.

Open your VS Code terminal and generate the project:

Bash

```
npx create-react-app eventexamplesapp
```

```
cd eventexamplesapp
```

2.2. Create EventExamples Component (`src/EventExamples.js`):Step 2.

Create `src/EventExamples.js` to handle counter operations, passing parameters, and synthetic click events:

JavaScript

```
import React, { useState } from 'react';
```

```
function EventExamples() {
```

```
  const [counter, setCounter] = useState(0);
```

```
  // 1a & 1b. Multiple actions in a single handler
```

```
  const handleIncrement = () => {
```

```
    setCounter((prev) => prev + 1);
```

```
sayHello();  
};
```

```
const sayHello = () => {  
  alert("Hello! Counter has been increased.");  
};
```

```
const handleDecrement = () => {  
  setCounter((prev) => prev - 1);  
};
```

```
// 2. Function receiving an argument  
const handleWelcome = (message) => {  
  alert(`Message: ${message}`);  
};
```

```
// 3. Synthetic Event Handler  
const handleOnPress = (event) => {  
  console.log("Synthetic Event Type:", event.type);  
  alert("I was clicked");  
};
```

```
return (
```

Counter & Event Examples

Counter: {counter}

```
{/* Increment invoking multiple methods */}
```

Increment

```
{/* Decrement */}
```

Decrement

```
{/* Button invoking method with 'welcome' argument */}  
handleWelcome('Welcome')}} style={{ marginRight: '10px' }}>
```

Say Welcome

```
{/* Synthetic OnPress button */}
```

OnPress

```
);  
}
```

```
export default EventExamples;
```

3.3. Create CurrencyConvertor Component
(`src/CurrencyConvertor.js`): Step 3.

Create src/CurrencyConvertor.js to convert Indian Rupees (INR) to Euros (EUR):

JavaScript

```
import React, { useState } from 'react';
```

```
function CurrencyConvertor() {
```

```
  const [rupees, setRupees] = useState("");
```

```
  const [euros, setEuros] = useState(null);
```

```
  // Fixed conversion rate (e.g., 1 EUR ≈ 90 INR)
```

```
  const EXCHANGE_RATE = 90;
```

```
  const handleSubmit = (e) => {
```

```
e.preventDefault();  
const inr = parseFloat(rupees);  
if (!isNaN(inr) && inr > 0) {  
    const converted = (inr / EXCHANGE_RATE).toFixed(2);  
    setEuros(converted);  
} else {  
    alert("Please enter a valid positive number for Rupees.");  
    setEuros(null);  
}  
};  
  
return (
```

Currency Convertor (INR to EUR)

Amount in INR (₹):

setRupees(e.target.value)}

placeholder="Enter amount in Rupees"

required


```
/>
```

Convert

```
{euros !== null && (
```

```
₹{rupees} INR = €{euros} EUR
```

```
))
```

```
);
```

```
}
```

```
export default CurrencyConvertor;
```

4.4. Update App.js:Step 4.

Update src/App.js to bring both components onto the home page:

JavaScript

```
import React from 'react';
import EventExamples from './EventExamples';
import CurrencyConvertor from './CurrencyConvertor';
```

```
function App() {
  return (
```

React Form Events Lab

```
);
}
```

```
export default App;
```

5.5. Run and Verify:Step 5.

Start the app in your terminal:

Bash

npm start

12-HANDS ON

1. Core Concepts

Conditional Rendering Techniques in React

Conditional rendering in React works the same way conditions work in JavaScript:

1. **if / else Statements:** Best used at the top level of a render() method or component function to return completely different JSX layouts.
2. **Element Variables:** Storing React elements inside variables (e.g., let button = <LoginButton/>;) allows you to conditionally compute part of the UI while keeping the main return statement clean.
3. **Ternary Operator (condition ? true : false):** Ideal for inline conditional rendering inside JSX.
4. **Logical AND (&&):** Used when you want to render a component only if a condition evaluates to true.

How to Prevent Components from Rendering

To hide a component completely, return null from its render function based on a condition. Returning null does not affect the firing of component lifecycle methods (like useEffect or componentDidUpdate).

2. Hands-on Lab: Ticket Booking App (ticketbookingapp)

In this lab, when the user is logged out (**Guest Page**), they can browse available flights. When they click **Login (User Page)**, they gain access to the ticket booking panel.

1.1. Create the React App:Step 1.

Open your VS Code terminal and initialize the project:

Bash

```
npx create-react-app ticketbookingapp
```

```
cd ticketbookingapp
```

2.2. Create Guest & User Components (`src/Components/`):Step 2.

Inside src/, create a new folder named Components.

Create **src/Components/GuestPage.js**:

JavaScript

```
import React from 'react';
```

```
function GuestPage() {  
  const flights = [  
    { id: 'AI-101', origin: 'Delhi', destination: 'Mumbai', time: '10:00 AM' },  
    { id: '6E-204', origin: 'Bengaluru', destination: 'Chennai', time: '02:30 PM' },  
    { id: 'UK-502', origin: 'Kolkata', destination: 'Hyderabad', time: '06:00 PM' }  
  ];  
  
  return (  

```

Flight Schedule (Guest View)

Please log in to book your tickets.

```
{flights.map((f) => (
```

-
- **{f.id}**: {f.origin} → {f.destination} ({f.time})
-

```
)))}
```

```
);
```

```
}
```

```
export default GuestPage;
```

Create **src/Components/UserPage.js**:

JavaScript

```
import React, { useState } from 'react';
```

```
function UserPage() {
```

```
  const [booked, setBooked] = useState(false);
```

```
  return (
```

Welcome, Logged-in User!

Select your flight to complete booking:

```
    setBooked(true)}
```

```
    style={{ padding: '8px 16px', backgroundColor: '#007bff', color:
'white', border: 'none', borderRadius: '4px', cursor: 'pointer' }}
```

```
    >
```

```
      Book Flight (AI-101)
```

```
{/* Inline conditional rendering using && operator */}
```

```
{booked && (
```

✓ Ticket Booked Successfully! Confirmation sent to your email.

```
})
```

```
);
```

```
}
```

```
export default UserPage;
```

3.3. Create Login / Logout Buttons

(`src/Components/Buttons.js`):Step 3.

Create **src/Components/Buttons.js**:

JavaScript

```
import React from 'react';
```

```
export function LoginButton({ onClick }) {
```

```
  return (
```

```
    Login
```

```

    );
  }

  export function LogoutButton({ onClick }) {
    return (

      Logout

    );
  }

```

4.4. Implement Conditional Rendering with Element Variables in App.js:Step 4.

Update **src/App.js** using **Element Variables** (button and pageContent) to conditionally switch UI components based on isLoggedIn state:

JavaScript

```

import React, { useState } from 'react';
import GuestPage from './Components/GuestPage';
import UserPage from './Components/UserPage';
import { LoginButton, LogoutButton } from './Components/Buttons';

function App() {
  const [isLoggedIn, setIsLoggedIn] = useState(false);

```



```
const handleLoginClick = () => setIsLoggedIn(true);  
const handleLogoutClick = () => setIsLoggedIn(false);
```

```
// Element variables for conditional rendering
```

```
let button;
```

```
let pageContent;
```

```
if (isLoggedIn) {
```

```
  button = ;
```

```
  pageContent = ;
```

```
} else {
```

```
  button = ;
```

```
  pageContent = ;
```

```
}
```

```
return (
```

Ticket Booking App

```
{/* Render element variable */}
```

```
{button}
```

```
{/* Render element variable */}
```

```
{pageContent}
```

```
);
```

```
}
```

```
export default App;
```

5.5. Run and Test the Application:Step 5.

Run the application in your terminal:

Bash

```
npm start
```

Open your browser at <http://localhost:3000>:

- **Initial View (Logged Out):** Shows the **Login** button and the **Guest Page** with the flight schedule.

- **Click Login:** Switches the button to **Logout** and displays the **User Page** with the interactive booking button.
- **Click Logout:** Returns seamlessly to the **Guest Page**.

13-HANDS ON

1. Core Concepts

Conditional Rendering Techniques in React

Conditional rendering in React works the same way conditions work in JavaScript:

1. **if / else Statements:** Best used at the top level of a render() method or component function to return completely different JSX layouts.
2. **Element Variables:** Storing React elements inside variables (e.g., `let button = <LoginButton/>;`) allows you to conditionally compute part of the UI while keeping the main return statement clean.
3. **Ternary Operator (condition ? true : false):** Ideal for inline conditional rendering inside JSX.
4. **Logical AND (&&):** Used when you want to render a component only if a condition evaluates to true.

How to Prevent Components from Rendering

To hide a component completely, return null from its render function based on a condition. Returning null does not affect the firing of component lifecycle methods (like `useEffect` or `componentDidUpdate`).

2. Hands-on Lab: Ticket Booking App (ticketbookingapp)

In this lab, when the user is logged out (Guest Page), they can browse available flights. When they click Login (User Page), they gain access to the ticket booking panel.

1

1. Create the React App

Step 1

Open your VS Code terminal and initialize the project:

Bash

```
npx create-react-app ticketbookingapp
```

```
cd ticketbookingapp
```

2

2. Create Guest & User Components (`src/Components/`)

Step 2

Inside src/, create a new folder named Components.

Create src/Components/GuestPage.js:

JavaScript

```
import React from 'react';
```

```
function GuestPage() {
```

```
  const flights = [
```

```
    { id: 'AI-101', origin: 'Delhi', destination: 'Mumbai', time: '10:00 AM' },
```

```
{ id: '6E-204', origin: 'Bengaluru', destination: 'Chennai', time:
'02:30 PM' },
```

```
{ id: 'UK-502', origin: 'Kolkata', destination: 'Hyderabad', time:
'06:00 PM' }
```

```
];
```

```
return (
```

Flight Schedule (Guest View)

Please log in to book your tickets.

```
{flights.map((f) => (
```

-
- {f.id}: {f.origin} → {f.destination} ({f.time})
-

```
)))}
```

```
);  
}
```

```
export default GuestPage;
```

Create src/Components/UserPage.js:

JavaScript

```
import React, { useState } from 'react';
```

```
function UserPage() {
```

```
  const [booked, setBooked] = useState(false);
```

```
  return (
```

Welcome, Logged-in User!

Select your flight to complete booking:

```
      setBooked(true)}  
      style={{ padding: '8px 16px', backgroundColor: '#007bff', color:  
'white', border: 'none', borderRadius: '4px', cursor: 'pointer' }}  
    >  
    Book Flight (AI-101)
```

```
    { /* Inline conditional rendering using && operator */}  
    {booked && (
```

✓ Ticket Booked Successfully! Confirmation sent to your email.

```
  )}
```

```
);  
}
```

```
export default UserPage;
```

3

3. Create Login / Logout Buttons (`src/Components/Buttons.js`)

Step 3

Create src/Components/Buttons.js:

JavaScript

```
import React from 'react';
```

```
export function LoginButton({ onClick }) {
```

```
  return (
```

```
    <button>Login</button>
```

```
  );
```

```
}
```

```
export function LogoutButton({ onClick }) {
```

```
  return (
```

```
    <button>Logout</button>
```

```
  );
```

```
}
```

4

4. Implement Conditional Rendering with Element Variables in App.js

Step 4

Update src/App.js using Element Variables (button and pageContent) to conditionally switch UI components based on isLoggedIn state:

JavaScript

```
import React, { useState } from 'react';  
import GuestPage from './Components/GuestPage';  
import UserPage from './Components/UserPage';  
import { LoginButton, LogoutButton } from './Components/Buttons';
```

```
function App() {  
  const [isLoggedIn, setIsLoggedIn] = useState(false);  
  
  const handleLoginClick = () => setIsLoggedIn(true);  
  const handleLogoutClick = () => setIsLoggedIn(false);  
  
  // Element variables for conditional rendering  
  let button;  
  let pageContent;  
  
  if (isLoggedIn) {  
    button = ;  
    pageContent = ;  
  } else {  
    button = ;
```

```
    pageContent = ;  
}
```

```
return (
```

Ticket Booking App

```
    { /* Render element variable */ }  
    { button }
```

```
    { /* Render element variable */ }
```

```
    { pageContent }
```

```
);  
}
```

```
export default App;
```

```
5
```

5. Run and Test the Application

Step 5

Run the application in your terminal:

Bash

```
npm start
```

Open your browser at <http://localhost:3000>:

- Initial View (Logged Out): Shows the Login button and the Guest Page with the flight schedule.
- Click Login: Switches the button to Logout and displays the User Page with the interactive booking button.
- Click Logout: Returns seamlessly to the Guest Page.